

ADVANCED PROGRAMMING LANGUAGES

SIMULATORE DI GUIDA

Casaccio Agrippino, matricola 1000085471

Il progetto che è stato sviluppato, denominato semplicemente “**SimulatoreAuto**”, un simulatore di guida sviluppato integrando i linguaggi di programmazione C++, Python e C#. Ognuno di essi è stato utilizzato per un compito specifico, in modo da sfruttare al meglio le loro caratteristiche principali e resi tra loro comunicanti tramite gRPC. Di seguito vi è una breve panoramica delle varie componenti.

- **COMPONENTE C++**

In C++ si è implementata la parte principale del simulatore, relativa al calcolo e alla gestione delle formule matematiche/fisiche.

A livello architetturale si sono sviluppati più file. In quelli header quale struct.h sono state dichiarate le strutture dati che rappresentano i valori del database, lo stato corrente del veicolo e i parametri fisici della simulazione; altri sono stati invece usati per separare la logica del motore, i calcoli fisici e la gestione generale del veicolo dal file principale.

Come precedentemente scritto, la comunicazione tra i diversi componenti è stata realizzata sfruttando canali gRPC, con il supporto a livello tecnico di Docker per la sua realizzazione. C++ sfrutta il metodo InvioCpp fungendo da server in modo da ricevere da C# la richiesta (e successivo avvio) di simulazione.

La classe Veicolo, ad ogni intervallo di simulazione, ne determina le caratteristiche del percorso (in base alla posizione attuale si determina se è in pendenza in salita o discesa. *A livello pratico ho provato a fare anche un percorso con curve ma il calcolo della forze mi è risultato troppo complicato e ho optato per semplici salite e discese*). Sono poi calcolate la direzione del veicolo e se vi è pendenza, per poi calcolare le forze agenti in quel momento quale quella gravitazionale, motrice, la resistenza al rotolamento ed aerodinamica, frenata e aderenza. Viene sfruttata la composizione, inglobando al proprio interno gli oggetti della classe Motore e Fisica.

Lapalissianamente, la classe Motore si occupa di quest’ultimo calcolando i giri al minuto a partire da velocità e trasmissione, effettua eventuali “cambi di marcia” e aggiorna la temperatura del motore. I parametri fisici sono stipati in una struct Coefficienti, mentre per i calcoli vettoriali (Vector3d) è stata importata e usata la libreria Eigen.

La classe Simulazione invece esegue la simulazione vera e propria. Il veicolo viene creato in modo dinamico tramite std::make_unique in modo da averne proprietà esclusiva ed evitare gestione manuale della memoria. Ad ogni iterazione aggiorna posizione, velocità, marcia, RPM e temperatura per poi inviarle a Python.

I dati dell'automobile sono stati recuperati (e precedentemente aggiunti) da un database MySQL sulla base delle indicazioni ricevute: in base alla macchina cambiano infatti massa, coppia, raggio della ruota, rapporti del cambio, differenziale e potenza massima. La connessione e gli oggetti associati come Statement sono gestiti tramite std::unique_ptr in modo da liberare automaticamente le risorse una volta che esauriscono il loro ciclo di vita.

Una volta effettuati gli opportuni calcoli e sempre mediante gRPC, i dati da Simulazione sono inviati tramite client PythonClient, mantenendo comunque privato lo stub e gestendo anch'esso tramite std::unique_ptr. Questo processo viene ripetuto ad ogni aggiornamento (vengono simulati 90s con passo di 0.5s).

- **COMPONENTE PYTHON**

In Python si è sviluppata la parte relativa alla raccolta ed elaborazione dei dati simulati sfruttando le librerie NumPy (per calcolo) e Matplotlib (per generare grafici).

A livello architetturale, viene avviato nello script main il server gRPC tramite ThreadPoolExecutor per la comunicazione con C++. Durante la simulazione descritta in precedenza, vengono inviati periodicamente i dati e tramite metodo InvioPy() vengono raccolti in un dictionary di list contenente: tempo, velocità, marcia, RPM, temperatura e coordinate dell’automobile.

Al termine della simulazione, col metodo FinePy() viene richiamato lo static method Telemetria.analizza() (è stato posto static perchè l’analisi eseguita non necessitava di accedere a dati o attributi di specifiche istanze), passandovi il dict con i dati.

Questa classe sfrutta le funzioni del modulo NumPy per eseguire diverse operazioni: la trasformazione delle list in array, calcolare velocità media e massima, l'accelerazione massima, la temperatura media, il numero massimo di RPM raggiunti, la marcia più usata e la distanza totale percorsa.

Sfrutta poi Matplotlib per produrre i grafici della traiettoria e dell'andamento della velocità. A livello puramente implementativo, per poter essere inviati mediante in modo più efficiente tramite messaggi gRPC, i grafici sono stati salvati direttamente in memoria tramite io.BytesIO e convertite in binario.

Il risultato di questa elaborazione è stato organizzato em mediante dictionary unpacking inserito nel messaggio ReportData da inviare al componente C#; sfruttando sempre canale gRPC stavolta come client.

Per una maggior sicurezza, prima dell'invio si è posto il blocco try/except per gestire eventuali errori, mentre tramite il modulo logging vengono registrati gli eventi accaduti in modo da poter avere dei feedback sul funzionamento. Al termine i dati vengono azzerati qualora dovesse essere eseguita una nuova simulazione.

• COMPONENTE C#

In C# si è implementa l'interfaccia grafica del simulatore sfruttando il framework Windows Forms.

A livello strutturale, la classe Form1 rappresenta la schermata principale dell'applicazione: costruisce dinamicamente gli elementi grafici quali pulsanti, label e immagini. All'esecuzione è possibile cliccare sui suddetti pulsanti per selezionare il modello dell'automobile (Panda e Golf) e le condizioni meteo del percorso da simulare (pioggia o sole con piu o meno il vento).

Sono stati sfruttati, a livello di codice, gli event associati alla selezione delgi elementi ivi descritti: quando si clicca su un pulsante infatti, viene attivato il relativo evento Click, il quale richiamerà il metodo per l'invio dei dati. Una volta selezionati, essi vengono disabilitati per impedire modifiche estemporanee e le informazioni relative inviati al componente C++ sempre mediante canale gRPC.

Per tale scopo, C# funge (analogamente agli altri) sia da client che da server: nel primo caso si occupa dell'invio a C++ dell'automobile e il meteo (i quali saranno poi usati per ricavare i dati dal database); nel caso server mediante PtoCService si occupa di ricevere il report finale da Python. La richiesta è effettuata tramite chiamata asincrona, in tal modo si è evitato di bloccare il thread GUI durante l'attesa.

Alla ricezione viene aperta una seconda finestra GUI, con la quale visualizzare il titolo della simulazione, i dati numerici e i grafici ricevuti. Questi ultimi vengono ricostruiti tramite MemoryStream, Bitmap e PictureBox. Solo dopo che il report verrà visulaizzato, è possibile tramite RESET riabilitare i pulsanti, in modo da poter effettuare una nuova simulazione.

Dato che il servizio gRPC può ricevere il report su un thread differente da quello utilizzato da Windows Forms, viene utilizzato il metodo Invoke per eseguirne l'aggiornamento su quello corretto. Infine, per quanto riguarda la gestione di risorse quali il canale gRPC e gli stream image, tramite using si sfrutta il meccanismo di IDisposable in modo da poter rilasciare le risorse automaticamente non appena non saranno piu necessarie.

